

Use of Whisper for translating spoken Cantonese to written Chinese

Introduction

I have chosen Whisper as the AI responsible for translating spoken Cantonese to written Chinese. Whisper is not perfect at translating the Chinese language and especially the Cantonese dialect because of the lack of existing samples used to train the base Whisper model. My goal is to determine if it's possible to improve Whisper effectively by training it.

I will also address the multiple ways of running Whisper and which Whisper model is the most efficient.

To Note: All scripts mentioned in the documentation runs code in the root folder [WHISPER_TRANSLATION]

The use of Hugging Face training model

Hugging Face is a very prominent AI training model that is used to train varies types of AI, which includes Whisper. I have decided to take a in depth look at how Hugging Face functions by trying its implementation.

In general, AI works by reading through a set of data that is segmented into training data and validation/evaluation data. Training data is used to train the AI, in which the AI will read through the training data and improve its performance in the process. The AI then performs on validation data which the AI has never seen before, so that the performance of the AI can be determined.

Hugging Face takes in Cantonese audio being segmented into 30 second clips, which are stored in .csv files; this procedure is done for both training and validation data.

```
wav_filename,transcript
D:\Programming_Stuff\Internship_2\TALKCT2021M08100005_01C_pt_1.wav,謝謝大家繼續支持《盤點政策》本集繼續訪問政務司司長李家超之前討論了不少關於政務司司長
D:\Programming_Stuff\Internship_2\TALKCT2021M08100005_01C_pt_2.wav,如何協調和統籌不同的政策局可舉出好幾個例子例如大家很關心的青年政策青年政策已有不少學
D:\Programming_Stuff\Internship_2\TALKCT2021M08100005_01C_pt_3.wav,必須協調很多不同的範疇包括例如教育、房屋以至整體的經濟發展政策和就業政策全部可能需要
D:\Programming_Stuff\Internship_2\TALKCT2021M08100005_01C_pt_4.wav,教育、經濟等各方面政策需要政務司司長統籌最後還有行政、立法關係立法會有那麼多問題向政
D:\Programming_Stuff\Internship_2\TALKCT2021M08100005_01C_pt_5.wav,好，立即繼續訪問李家超司長我們上星期來到政務司司長辦公室與李家超司長討論行政、立法關
D:\Programming_Stuff\Internship_2\TALKCT2021M08100005_01C_pt_6.wav,已沒有時間做事官員是否可以輕鬆度過那幾個月？我想怎麼可能？不可能的但我想問司長一事關
```

Figure 1.csv file for some Training data

A tokenizer and processor is then defined for preparing the inputs of the Whisper model and processing the inputs respectively.

A common determiner will be used to indicate the performance of the trained Whisper model when it reads through validation data. For now, WER is used to determine its performance, which stands for “Word Error Rate”. Other types of determiners will be discovered upon later.

Downloading Applications to Run the Scripts:

Whisper is mainly based on the processing power of CUDA drivers which exist in NVIDIA GPUs. The driver must be needed to run [Whisper X](#). I personally used [CUDA 11.8](#), but CUDA 12.x should work too.

I have attached requirements.txt in the folder, which shows the installed packages needed to run all the scripts using CUDA 11.8. If CUDA 12.X are used, versions of packages may differ.

After installation, to make sure Whisper X runs, make sure cudnn_cnn_infer64_8.dll, cudnn_ops_infer64_8.dll, cudnn_adv_infer64_8.dll exists in the CUDA bin file (bin file usually exists in C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v1x.x\bin for Windows)

There may be some issues or conflicts in packages that I wasn't able to test out, but [this link](#) should help regarding problems with ctranslate2 and faster_whisper package conflicts.

How to use the Hugging Face Trainer

Use of Datasets

In general, the Hugging Face trainer works by taking in a set of inputs [e.g. Cantonese voice clips or Cantonese words] and uses some methods to wrap these inputs into a format that can be read by the trainer. The wrapping of inputs is typically done using Tokenizers from the Transformers library. The inputs are then processed by the chosen AI model, and the model will learn from the inputs.

Hugging Face trainer can take in inputs using different methods and is completely based on how the trainer is set up. I used two methods of doing so.

Written Cantonese to Written Chinese

[raptorkwok/cantonese-chinese-translation]

(This is deprecated and is no longer attempted)

With a very direct language to language translation model, data is taken in by using datasets that can be uploaded in Hugging Face. From my experience, most Hugging Face translation datasets uses json file(s).

My json file used for training looks like this:

```
{
  "id": "0",
  "translation": {
    "yue": "歡迎大家收看盤點政策我哋同大家一齊精讀下唔同政策局以至成個特區政府上上下下過去呢幾年做嘢做點",
    "zh": ""
  }
},
{
  "id": "1",
  "translation": {
    "yue": "我哋希望由所謂解困新聞行嚟角度睇如果你做唔嚟有啲咩限制呢由於咁咩原因呢啊嚟近呢幾個月追唔追得返個進度呢定係嘅時而勢易喇要計",
    "zh": ""
  }
},
{
  "id": "2",
  "translation": {
    "yue": "離任期之內係咪真係可以趕得切做嚟所有嘢呢我係黃永我係陸漢文 Benson 好喇咁我哋今日呢就唔好淨係睇一份死刑報告喇 Benson 係因",
    "zh": ""
  }
},
{
  "id": "3",
  "translation": {
    "yue": "由一份帶到落去下一份都有幾多話同嚟講題個啲個真係政策連貫性都好緊要喇咁咁如果我哋一路推推推推推到去第一份施政報告已經係講",
    "zh": ""
  }
},
{
  "id": "4",
  "translation": {
    "yue": "啊特首喺立法會又好似都唔係留咗好耐喇咁嚟係喇喇嚟以往嚟特首呢都會係逐次宣讀呢但嚟嚟施政報告嚟不過林太呢佢就話嚟嚟行政會議",
    "zh": ""
  }
},
{
  "id": "5",
  "translation": {
    "yue": "主要就講一啲大方向嚟嚟係喇然後做嚟嚟呢就係即係相對厚啲嚟讀嚟份文字版喇或者當然有上到嚟嚟師範報告度啲嚟錯喇咁咁如果係咁呢份",
    "zh": ""
  }
}
```

A lot of datasets are available on Hugging Face, but you can upload them yourself.

Uploading Dataset onto Hugging Face for Use The Written Cantonese to Written Chinese is no longer attempted because unlike Whisper, Raptorkwok has no logic in deducing that actual meaning of the sentence.
Cantonese Voice Clips to Written Chinese Cantonese voice clips are unlikely to be placed in datasets in the traditional format as mentioned above. What I did was to use a .csv file that stores the file path of the 30 second voice clips (the Whisper models doesn't take voice clips over 30 seconds) and the corresponding Written Chinese Text. For example, for my .csv, it looks like this: wav_filename,transcript D:\Programming_Stuff\Internship_2\TALKCT2021M08100005_02C_pt_1.wav,來到《盤點政策》第二節剛才上一節談到展望未來在 D:\Programming_Stuff\Internship_2\TALKCT2021M08100005_02C_pt_2.wav,是否藉此排除異己？是否要管制大眾，不准批評政府 D:\Programming_Stuff\Internship_2\TALKCT2021M08100005_02C_pt_3.wav,越讓我們有更多自由的空間否則便會弱肉強食在資訊

Execution Training for Hugging Face Using Different Types of Whisper:

Hugging Face accepts many types of Whisper models for training. My plan is to test on Whisper “Large” models: “whisper-large-v2” and “whisper-large-v3” along with their distilled counterparts. Whisper “Base” and “Large” models are trained on multiple languages instead of just English, so they should be more competent at achieving the transcription.

I present two methods on running Whisper, both with its pros and cons. With that said, both methods require the use of **Audio_Text_Processing.py**

Note: all scripts should be run when user is at the root folder [WHISPER_TRANSLATION] (refer to image below)

Audio_Text_Processing.py This script takes in full length audio files in the Unsplited_Audio folder, and SRT file texts in the SRT folder. Make sure every audio has its own SRT file , or else weird behaviour will show. When the script is run, each audio file will be split based on the timestamps in the SRT file and the specified length of each time segment (30 seconds). The Chinese text in the SRT file will also be saved into sentences based on the time segments. In the end, a csv file will be produced at the CSV_file folder. The csv contains the file locations of the split audios, and the corresponding written Chinese text from each split audio. Remember to clear the Auto_Split_Audio folder for each re-run. Remember to delete the CSV_file before re-run if duplicate training contents is not desired

Method 1: The conventional method

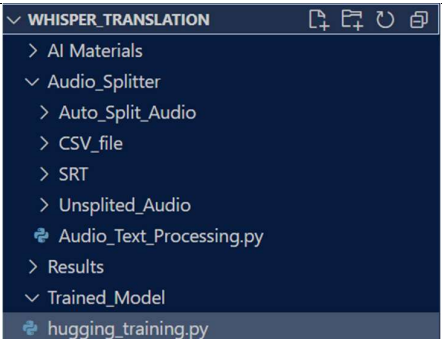
This method is the usual way people fine-tune whisper. This method allows evaluation to be done after a certain number of steps, which allows the user to see the change in evaluation output between steps. This allows debugging the training output easier.

This method should only be used if you have a fast-processing GPU with a large amount of RAM. Obviously, different types of Whisper Models occupy varying RAM sizes. Personally, my 3070Ti Nvidia GPU with 8 GB RAM is only sufficient in fine-tuning models like “Whisper-Base”, but not the Large Models [e.g. “Whisper-Large-V2]. The large models fail due to the limited RAM storage.

The company's 4090Ti 24GB RAM can effectively train the Large Models with this method.

Using this Trainer

Note: all scripts should be run when user is at the root folder [WHISPER_TRANSLATION] (refer to image below)

Hugging_training.py	
This script reads the CSV file from the CSV_file folder. If CSV_file folder is empty, make sure to run Audio_Text_Processing.py first. Purpose of the CSV file is for the training model to read audio file paths and corresponding written Chinese text. Each time this script's run is completed, a new fine-tuned model will be created at Trained_Model for use.	

Method 2: The “efficient” method

This method is a much more efficient fine-tuning method compared to the first method. It is basically the same method, but with the additional use of the PEFT LoRA model (Parameter-Efficient Fine-Tuning, Low-Rank Adaptation). The PEFT model only trains the fine-tuning parameters that are important to the results instead of all the parameters, which reduces processing time and the need for GPU RAM, while maintaining performance of the training process. Therefore, this method should be used if your GPU RAM is limited.

However, this implementation only evaluates after training is completed. Therefore, viewing evaluation between training steps is not possible. You can still compare the initial performance and the final performance though. Other than that, this method should also be under-performed compared to the first method, but the difference is only about 1% WER.

Note: all scripts should be run when user is at the root folder [WHISPER_TRANSLATION] (refer to image below)

hugging_training_optimized.py
Just like the first method, you can choose to train different kinds of Whisper Models. This can be changed in the model_name variable in the file. After choosing the model, run this script file. This script only runs the “train” part of the fine-tuning process. If the full script is run completely, two new models will appear: 1. Base Model: This model will appear in the Base_Model folder. This model is the initial model that is used before training. 2. Trained Model: This model will appear in the Trained_Model directory. This works similarly to the previous trainer (Hugging_training.py)
hugging_training_optimized_eval.py
This script only runs the “evaluation” part of the training process. Remember to always run “hugging_training_optimized.py” first for training. This script reads model parameters from the two types of models listed above and feeds it into the peft_evaluation function for evaluation. Choose to evaluate only the models created by hugging_training_optimized.py. <pre># This variable should be set to the directory of the resulting models after running hugging_training_optimized.py model_name = "Trained_Model/Whisper-Large-v2-noisy" # (Trained_Model/xxxxx or Base_Model/xxxxx)</pre>
Same as previously, a SRT file will be produced in the SRT_Produced folder

Installing Libraries for Training

Note: When running the scripts on a device for the first time, install the dependencies required.
[I personally use pip install, but there is more than one method to do so]

However, for using **torch utilities**, **do not use “pip install torch”** because it defaults to the no CUDA version of torch, which forbids the trainer to use GPU. **Follow <https://pytorch.org/get-started/locally/> to generate a custom torch installation which contains CUDA options.**

Find your CUDA version of your GPU by typing this command in your terminal: **nvidia-smi**

When using Whisper Large V3, if transformers cannot detect the “yue” (Cantonese) language, upgrade transformers using command “**pip install git+https://github.com/huggingface/transformers**”

Using the Auto-Trainer class

```
training_args = Seq2SeqTrainingArguments(  
    output_dir='./Results',  
    logging_steps=2,  
    evaluation_strategy="steps",  
    learning_rate=1e-3,  
    per_device_train_batch_size=4,  
    per_device_eval_batch_size=4,  
    weight_decay=0.01,  
    save_total_limit=3,  
    num_train_epochs=45,  
    warmup_steps=20,  
    save_steps=20,  
    eval_steps=4,  
    gradient_accumulation_steps=1,  
    push_to_hub=False,  
    report_to=["tensorboard"],  
    metric_for_best_model= "eval_bleu",  
    greater_is_better=True,  
    predict_with_generate=True  
)
```

For any type of hugging face training, Trainer classes can be used.

Seq2SeqTrainingArguments differs from the regular TrainingArguments class in a way that its suited for language translation and summarization tasks.

Manually constructing training procedures is possible, but using the classes are much simpler.

Below explains more about the available arguments

Note: Difference between Epochs and Steps

Steps: Each step is a gradient update, and it means that “batch size” amount of samples are processed	Epochs: Each epoch refers to one full cycle through the training data.
Warmup_steps: The number of steps where the trainer slowly starts accumulating learning rate. Model learns incredibly slowly at the start of Warm Up and will learn at the specified learning rate at the end of Warm Up.	per_device_train_batch_size/ per_device_eval_batch_size: The number of batches of data each available GPU/CPU core can take in for processing. Higher value means higher parallel processing, but will take more memory Defaults to 8 if not set
gradient_accumulation_steps	predict_with_generate: Alters forward pass of the AI trainer, “produces

Loops over your forward and backward pass. Using this saves memory, but limits the use of parallel processing. Use this when whole batch size doesn't fit into one GPU. Defaults to 1 if not set.	text sequences during evaluation, which can then be compared with the ground truth sequences to compute these metrics.” This should always be set to true for all translation tasks
save_steps Number of updates steps before two checkpoint saves. Lower the number, more the checkpoints	Eval_steps Number of steps until an evaluation is made. [The use of test data to test for performace]
evaluation_strategy Choose the use between steps or epochs. Note that if epochs is selected, eval_steps will be redundant	
gradient_checkpointing Reduces training speed, but can be used to free a lot of memory	metric_for_best_model Specifies what metric is used to compare two different models.
load_best_model_at_end If set to true, will save the model with the best checkpoint after executing training	Generation_max_length control the length of the generated output to avoid excessively long or short sequences
report_to if report_to=["tensorboard"] is set, you can inspect the training metrics of your train by invoking the command : “ tensorboard --logdir ./Results ”	

Different Types of Determiner Performances (Comparing WER and BLEU)

It has come to my realization that using the WER is not the only way to determine if the converted text is acceptable,

I came across a website <https://dl.acm.org/doi/fullHtml/10.1145/3639233.3639332> which gave me more insight. I will now examine the use of BLEU.

BLEU	BLEU works based on the presence of n-grams in the reference translations. N-grams work by segmenting each sentence in data into individual characters and will compare the characters between the correctly written sentence of the data and the sentence generated by Whisper. BLEU specifically does this in 4 ways: Unigram, Bigram, Trigram and Quadigram, which refers to separating sentences into one, two, three or four characters respectively. For example, with the sentence 已沒有時間做事官, it will be segmented as follow: Unigram: [已,沒,有,時,間,做,事,官] Bigram: [已沒,有時,間做,事官] Trigram: [已沒有,時間做,事官] Quadigram: [已沒有時,間做事官]
------	--

	BLEU will compare the generated strings with the n-grams, to detect if phrases that exist in the n-grams exist in the generated strings. The BLEU score will be high if many phrases in
WER	WER is concerned about the number of insertions, deletions and character replacements compared to the original string. This works better for English because of its fixed structural nature (e.g. [noun + verb + adjective + noun]), but it isn't the same for Chinese. The problem is further amplified because subtitles do not usually come in full sentences, but rather in short phrases that conveys a meaning.

In my code implementation, edit the use_WER variable to switch between the use of BLEU or WER:

```
# if use_WER is False, BLEU will be used as evaluation metric
use_WER = False
```

In `hugging_training.py`

Comparison of Model Types

Model Memory Requirements

From trial and error, I realised some models cannot be run without sufficient GPU RAM. For example, my laptop uses a 3070ti GPU with a 7.8GB Dedicated Memory (RAM). Processing power wise, it is sufficient to run Whisper Large Models, but the GPU RAM is not enough to run it.

From Whisper's official Github page, I found a diagram that explains the minimum memory usage for each general model type.

Size	Parameters	English-only model	Multilingual model	Required VRAM	Relative speed
tiny	39 M	<code>tiny.en</code>	<code>tiny</code>	~1 GB	~32x
base	74 M	<code>base.en</code>	<code>base</code>	~1 GB	~16x
small	244 M	<code>small.en</code>	<code>small</code>	~2 GB	~6x
medium	769 M	<code>medium.en</code>	<code>medium</code>	~5 GB	~2x
large	1550 M	N/A	<code>large</code>	~10 GB	1x

This explains why the company's 4090Ti GPU can train Whisper Large Models effectively; its because it has at least a 16 GB RAM.

Memory can be managed to some extent. Here are some methods:

1. Reduce Training and Evaluation Batch Sizes and increase gradient accumulation
2. Use Optimizers in training arguments
3. Remove Torch Cache using `[torch.cuda.empty_cache()]`

Model Memory Limitations

From my observations after fine-tuning previously fine-tuned models, it came to my attention that the resulting model has the same file size as the previously fine-tuned model. This could indicate that while the model learns new words, it could potentially forget other words.

Distilled Whisper models vs Base Whisper models

I learnt more about how Distilled Whisper models differ from the Base models. Distilled Whisper models run up to 6 times the speed of Base Whisper models but run the risk of having more problematic mistakes such as missing sentences. Based on the article shown in the link below, distilled models do not differ in quality too much from base models, and sometimes can even out-perform Base models in terms of punctuation. This will be tested eventually.

<https://hub.zenoml.com/report/1123/Comparing%20OpenAI%20Whisper%20Transcription%20Models>

Expectingly, Distilled Whisper models require less resources to train compared to its Base model variants. For example, Distilled Whisper Large V2 has a minimum requirement of a RTX 2080 Ti GPU, while Base Whisper Large V3 has a minimum requirement of a RTX 3060.

It however does seem that Distilled Whisper models do not support non-English languages, so it might produce terrible results. This will be tested.

Whisper Large V2 vs Whisper Large V3 models

There were a lot of debate on whether Whisper Large V2 or Whisper Large V3 is considered a better model. From Whisper's official description, they claim that it produces 10 to 20% less errors, a different spectrogram (sound-waveform). However, other users claim that it has higher amounts of hallucinations and lower accuracy. This will be tested.

Whisper Large V3 also has a new Cantonese language identifier, which could be useful. This will also be tested upon.

Fine-Tuning Tests

Train Parameters for Whisper Base

In the training segment, I am going to use the same training metrics for all the models listed below. The training metrics are shown on the right.

You can always refer to [Using the Auto-Trainer class](#) to have a better understanding of the Auto-Trainer class settings.

```
training_args = Seq2SeqTrainingArguments(  
    output_dir="./Results",  
    per_device_train_batch_size=16,  
    gradient_accumulation_steps=4,  
    learning_rate=1e-5,  
    warmup_steps=20,  
    num_train_epochs=50, # Train for 10 epochs  
    gradient_checkpointing=True,  
    fp16=True,  
    # evaluation_strategy="steps", # Evaluate every 10 steps  
    evaluation_strategy="epoch",  
    # eval_steps=20,  
    per_device_eval_batch_size=2,  
    predict_with_generate=True,  
    generation_max_length=225,  
    save_steps=20, # Save model every 10 steps  
    report_to=["tensorboard"],  
    metric_for_best_model="wer",  
    greater_is_better=False,  
    push_to_hub=False,  
)
```

These tests are used on data provided by the 《盤點政策》 series. 8 episodes worth of audio were used, which 80% of the audio is used for training, and 20% of the audio is used for evaluation: This totals 193 segments of 30 seconds audio.

All the tests below (except for the WER vs BLEU comparison) uses WER (word error rate) as the training metric. [More about WER](#)

Before looking at the tests, I have recorded the approximate time needed to fine-tune each model.

Model	Time
Whisper-Base Model	About 30 minutes (without PEFT LoRA) [both clips]
Whisper-Large-V2 Model	About an hour (with PEFT LoRA) [盤點政策] About 2 hours (with PEFT LoRA) [Hipsters] [unknown without PEFT LoRA]
Whisper-Large-V3 Model	About 2 hours (with PEFT LoRA) [unknown without PEFT LoRA]

Whisper-Base Model

Result: Bad

The Whisper-Base Model is a relatively small Whisper model which is trained for 680k hours on multilingual data. Based on parameter metrics, this is the 2nd smallest official Whisper model, with only “whisper-tiny” being smaller.

I ran the model using the training parameters and data, and found slight improvements in the model after training, but it is not significant. For example:

Original Sentence: 他要了解產業、旅遊和文化，還有體育會否設定時很理想卻未必找到合適人選？其實...林太，哪有人甚麼都懂？並不需要全都懂得身處局長的層次負責每個決策局都要有一份動力和熱情就是我們常說的 **Passion**

First Iteration: 又要適產業 又要適女 又要適民化還要適看曲會否設定時你很理想但未必找到人做其實... 哪有人弄得也會加拿大所以說也會去到這個層次其實做誰的決策局需要有份動力有份熱情 我就說是專業
Final Iteration: 又要適產業、適旅遊、穩穩民化還要適合體育在設定時你很理想但未必能找到人做其實哪有人要，也要適合思考所以有兩種適的人去到層次做哪個決策局需要有份動力有份熱情，我常說是專門

I have highlighted significant differences between the first evaluation and the last evaluation of a sentence in the dataset. Even though some parts of the final iteration did improve compared to the first iteration, the final iteration result is still faulty.

From my observation, Whisper-Base is trying too hard to make sure everything makes sense and is too strict about it. There are situations where the AI will either completely ignore names of people or places or will try to change the names into actual words that doesn't make sense. For example:

Original Sentence: 好，來到節目第二節繼續與政務司司長李家超討論行政
Final Iteration: 好，回到這節，繼續與穩小思長的立法關係討論討論行政

Of course, it sometimes still works:

Original Sentence: 也是全球關注的議題因此這次與黃錦星局長談談到底有否一些政策還沒實行有否一些成了「政策大咗鬼」？場面一轉，現在來到政府總部東翼歡迎環境局局長這次非常感謝 KS，平日大家稱你為 KS
Final Iteration: 也就是全球關注的議題所以今天要與黃錦星局長談談究竟他做了沒有政策究竟他沒有政策究竟他沒有政策究竟他沒有政策....

Another major issue is that Whisper-Base always uses its own logic to make sentences more “simplistic”, but ends up breaking the logic of the sentence as a whole.

Original Sentence: 可以考慮興建過渡性能源發電設施？例如某幅土地空置數年若有足夠回本期是否可考慮加建太陽能發電板？
Final Iteration: 也能想法過渡性能源發電設施例如那篇計劃在幾年能否回回既足夠，能否試過想法輻發電版？

Other problems that commonly appear are:

- Chinese words are turned into English Words
e.g. 太陽能 → Tireland
- English Words are completely misspelt
e.g. Leading bureau → Bero

Whisper Large V2 Model

Result: Good

The Whisper Large V2 Model is a much larger Whisper model compared to the base model. As previously stated, I am using [method 2](#) to run the large models.

As you can see below, the translation of the model even before the training makes much more sense than the Whisper Base Model. There are also much less hallucinations compared to the Whisper Base Model.

Training the model makes the output align closer to the format of the original sentence. Looking at the corresponding colours of the sentences below, you can see the output after training looks more similar to the format of the original sentence compared to how the output looks before training is performed.

Original Sentence: 希望可以講解給觀眾理解其實有很多幕後的事議員所謂務實做事最後都是怎樣令到成為公共行政或公共服務的一部份所以我們慢慢再講 Anita 你對於剛才司長見到第一日開會就寫信你好像很緊張 為甚麼因為覺得其實這件事都幾特別譬如普通的觀眾看就會想知道
Before Training: 希望可以為觀眾講解很多幕後工作議員務實做事最後如何成為公共行政或公共服務一部分因此我們慢慢討論 Anita，你對於剛才司長的分享看到他首天上任便發信你似乎很緊張，為甚麼？因為覺得這件事挺特別例如普羅觀眾看到就想知道
After Training: 希望能講解給觀眾理解其實有很多幕後工作議員無實做的工作最後令到成為公共行政或公共服務的一部分所以我們慢慢再說 Anita，你對於剛才司長看到第一天開會便要寫信你好 像很緊張，為何？因為我覺得這挺特別普通觀眾看了便會想知道

It doesn't work well for all sentences though. I found a trend for these translation errors. These translation errors stem from sentences being way too long than it should. I found out that the concatenated sentences sometimes do not have punctuation between segments. This may cause confusion to the AI.

Original Sentence: 他要了解產業、旅遊和文化，還有體育會否設定時很理想卻未必找到合適人選？其實... 林太，哪有人甚麼都懂？並不需要全都懂得身處局長的層次負責每個決策局都要有一份動力和熱情就是我們常說的 Passion ← sentence without punctuation
Before Training:

既要懂產業，既要懂旅遊，又要懂文化還要懂體育在設定時很理想但未必找到工作？誰這樣都懂？林太，誰都懂到這層次時做哪個決策局，需要有一份動力有一份熱情，我們常說 Passion

Even if we take hallucinations away from the picture, the transcription is still worse than the Whisper Large V2 model.

Distilled Whisper Models

Result: Terrible

Distilled Whisper is a proclaimed cost-effective version of the original Whisper models, which is “6 times faster, 49% smaller, and performs within 1% word error rate (WER)”.

From my experience with training the Whisper Large V2 of Distilled Whisper, it has come to the conclusion that it is absolutely horrendous for our task. As previously stated, Distilled Whisper is not multilingual, which means it is default to expect the input language to be English, and it shows.

Noisy Dataset Tests

《盤點政策》was used in the previous tests above. However, due to the lack of background noise in 《盤點政策》 episodes, the capability of Whisper’s transcription on audio with background noise/music cannot be tested.

Therefore, as a workaround, I used another dataset from the “Hipster” programme. “Hipsters” consist of a lot of music in the background when people speak. This test is to show the effectiveness of the fine-tuned models on audio that could potentially be blurred by music.

Whisper Base Model

Result: Similar to [Whisper-Base without noise](#)

With this particular test, I am using audio clips from “Hipsters”, an RTHK production series. The difference between this and 《盤點政策》 is that “Hipsters” consist of a lot of music in the background when people speak. This test is to show the effectiveness of the trainer model on audio that could potentially be blurred by music.

From a general observation, the Whisper-Base Model with noisy audio fine-tuning works on par with the initial [Whisper-Base Model](#) test

An example would be

Original Sentence: 本集《潮青誌》，我們有甚麼內容？有「超 Chill」介紹復古時裝中最常見的風格，摩登造型帶大家一同回到六十年代「青春頌」一班年輕人就分享交換禮物的趣事到底大家收到最浮誇最不喜歡禮物有哪些？還會有香港摔角手向大家介紹這項冷門運動「新時代奮鬥者」黃寶瑤帶來歌曲《習慣孤單》
1 st Iteration: 今集， 朝清智 ，我們又會說甚麼我們有 超越 介紹回家伏股市中最常見的風格， 摸燈造型 跟大家一起回到 60 年代 青春頌 還有一群年輕人分享家換禮物的 吹嘢 到底大家收到最 逃勻 和最不喜歡的禮物有甚麼呢？還會有 香港叔叔 的手跟大家介紹這項 能玩 運動和新時代分者黃寶姚大尼哥曲集關孤單
50 th iteration: 今集《潮青誌》我們又會講點甚麼我們有 超主 介紹返復古時中的最常見的風格、摩登造型跟大家一起六十年代 青春頌 一班年輕人分析一下交換禮物的 趨勢 讓大家收到最 逗勞 還有最不喜歡

的禮物有甚麼呢還會有香港疏國手說明跟大家介紹這項能滿運動及新時代分製黃寶瑤大來歌曲《雜貫孤單》

WER or BLEU?

As stated before, WER is not a very good evaluation metric. In this test, we will see if using BLEU would really cause a significant difference in the training result. For this test, the Whisper-Base model will be used for convenience, and on the noisy dataset “Hipsters”.

The results show that by absolute technicality, BLEU does worse than WER. However, there honestly isn’t much of a difference, since almost all evaluation sentences contain the same results, with only minor exceptions.

Exceptions where WER performs better than BLEU:

Original Sentence:

原來生活習慣不同旅行時會發生很大衝突我試過有一次自己去台灣第一次獨自去台灣我住在背包客的床位...青年旅舍 對認識了幾個女孩子，然後一起去玩在旅舍認識，然後馬上一起去玩奇遇很有趣

Final iteration using WER:

原來生活習慣不同旅行時會發生的風格去新冠關係很大衝突我試過有一次自己去台灣第一次獨自去台灣我住在背包客的床位...青年旅舍認識了幾個女孩子然後一起去玩在旅舍認識的旅舍去玩奇遇？有趣

Final iteration using BLEU:

原來生活習慣不好格格，原來發現很大衝突去生活時發生發生發生甚麼關係我試過有一次自己去台灣台灣第一次自己去台灣台灣會在背包客通常是床位...青年旅舍兩年多旅舍認識了幾個年旅舍？然後一起去玩在旅舍認識、去玩在旅舍認識奇遇？有趣

This result could be only due to this dataset, there might be a chance that BLEU could outperform WER on other datasets. All and all, difference is not significant.

Running Whisper

General knowledge of running Whisper

Running Whisper is less costly than fine-tuning Whisper, and it takes way less time. All types of Execution Methods listed below uses less than 2 minutes to process a 23 minute RTHK clip.

In general, a model will be loaded, either from OpenAI, or from a fine-tuned model in a repository or local directory. If my fine-tuning methods are used, a fine-tuned model will be saved in the “Trained_Model” directory.

<pre>> run_whisper ✓ Trained_Model > whisper_large_v2 > whisper_large_v3_yue > whisper_large_v3_zh</pre>	<pre># Reference openai model_id = "openai/whisper-large-v3" # Reference from Directory model_id = "./Trained_Model/whisper_large_v3_zh"</pre>
--	--

There are a couple of directories under the “run_whisper” directory, which contains all the components that is used to run whisper on RTHK video files.

Video_Files	Place the RTHK videos that needs subtitle transcription here.
Processed_Videos	After processing and generating an SRT file, the video will be moved from Video_Files to here
SRT_produced	The SRT files generated will be placed here
wav_converted	Ignore, this is just used by the program

I created two different files for running the different types of Whisper execution methods shown below.

run_whisper.py This python script is for running the “Basic Execution Method” and the “Segmentation method”. You can easily choose between the two methods by changing a variable as shown below. <pre>''' Users can choose a model to run either from the Trained_Model local directory or load from the Hugging Face model hub. Modifying the variables above the line is enough ''' # Load from HuggingFace openai # model_id = "openai/whisper-base" # Reference from Local Directory model_id = "./Trained_Model/1" # if use_whisper_normal is False, whisper_segmentation method will be used use_whisper_normal = True</pre>
run_whisper_x.py This python script is for running the Whisper X method. Choose the model to be run by modifying “model_id” <pre>import whisperx import gc import os import shutil ''' You only need to specify the directory containing the fine-tuned / Open-AI model Modifying the variables above the line is enough ''' model_name = "./Trained_Model/14"</pre>

Types of Execution Methods

Basic Whisper Execution

This is the conventional method of running Whisper.

Assuming there are videos in the “Video_Files” folder, a video file will be processed and then moved to the “Processed_Videos” folder from the folder when the program is run once. An SRT file will be produced in the “SRT_produced” folder as a result.

A major limitation of this method is its faulty timestamps. The timestamps are initially correct, but the start time and end time of each segment will eventually become very similar. An example is shown below:

<pre>373 00:19:28,880 --> 00:19:31,200 忘記了動作 374 00:19:32,360 --> 00:19:32,440 我也不知道有甚麼問題 375 00:19:34,360 --> 00:19:34,440 但喜歡的音樂很重要 376 00:19:35,440 --> 00:19:35,520 首先要聽到 377 00:19:37,400 --> 00:19:37,480 人家特意在街上學習 378 00:19:38,120 --> 00:19:38,200 很活動</pre>	I am not convinced 10 words can be read in less than 0.1 seconds. The start and end times are clearly wrong starting from line 374.
--	--

Another problem is Whisper’s hallucination. Hallucination occurs during silent/speechless audio, where Whisper doesn’t know how to process the input, and therefore outputs random gibberish during those segments. This usually occurs during the start and end of a video. An example is shown below:

<pre> 0 00:00:00,000 --> 00:00:03,280 【本節目內所有玩意都請承認】 1 00:00:03,360 --> 00:00:04,360 謝謝 2 00:00:04,440 --> 00:00:05,760 我們會繼續帶給你 3 00:00:05,840 --> 00:00:08,720 全港第一代高級玩具 4 00:00:08,800 --> 00:00:11,440 以及全球第一代高級玩具 5 00:00:11,520 --> 00:00:13,440 全球第一代高級玩具 6 00:00:13,520 --> 00:00:15,440 全球第一代高級玩具 7 00:00:15,520 --> 00:00:17,920 全球第一代高級玩具 8 00:00:18,000 --> 00:00:19,480 本星期《朝青節》 </pre>	There is no speech in the video until the 18 second mark. Anything before that is hallucination (line 0 to 7)
---	--

Pros	Cons
- Rarely misses any sentence during transcription - Processes whole video at once (Whisper knows full context of audio, and therefore can predict more accurately)	- Faulty Timestamps - Huge amounts of hallucinations at the start and end of a video

Segmentation Method

I developed a custom algorithm that solves the hallucination problem, but then creates some new problems. It makes this method somewhat worse than the basic execution.

This method involves using a module “silero_vad”. The idea is that silero_vad can segment out the audio into multiple parts that involves speech; any speechless audio is discarded. This eliminates the hallucination issue completely but leads to many other issues.

1. Less context for each audio clip:
Whisper predicts audio based on the context of the audio clip. This implementation benefits less from this because audio is segmented into multiple small audios, and a new Whisper instance is run on each small audio clip once.
2. Worse Faulty Timestamps than the Basic Whisper Execution:
As stated previously, this occurs near the end of an audio clip. This approach leads to many audio clips, which also means there are many audio end segments.
3. Unclear speech audio may be segmented out:

Silero_vad uses audio thresholds to determine if the audio has speech or not, and it isn't perfect; some blurry speech audio may not be regarded as speech and is discarded.

Pros	Cons
- No hallucinations	- May miss some unclear sentences - Faulty Timestamps

Whisper X

Whisper X is an improved and modified Whisper algorithm that has VAD (voice activity detection) and Speech Diarization (the ability to identify individual speakers). This algorithm has, in current knowledge, solved almost all problems. (More about [whisperX](#))

VAD and Speech Diarization have proven to be effective at minimizing hallucinations without the need to segment audio in any way. The timestamps are also fixed.

If there are issues when running Whisper X,

The only potential issue is that the length of each line transcribed is very long. An example is shown below:

0 00:00:22,193 --> 00:00:49,189 今集潮青節,我們又會說甚麼我們有Chiu Chiu介紹復古時裝中最常見的風格,摩登造型跟大家一起回到60年代青春中的一群年輕人分享交換禮物的趣事到底大家收到最浮誇和最不喜歡的禮物甚麼呢還會有香港摔角手為大家介紹這項冷門運動以及新時代奮鬥者黃真意帶來的歌曲《習慣孤單》 1	The sentence is slightly longer than 18 letters. Editor will need to use whatever SRT application they have to manually segment the sentences.
--	--

Pros	Cons
- Less hallucinations - No Faulty Timestamps - Won't miss sentences	- Each sentence is extremely long

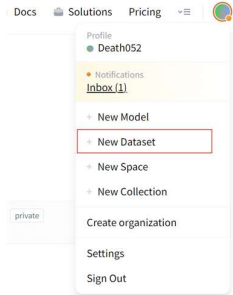
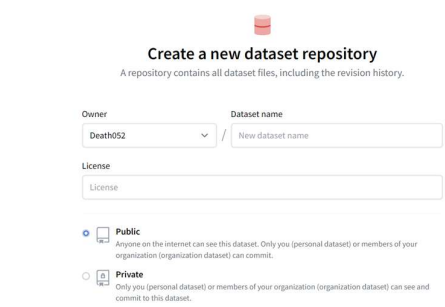
Fine-tuned models from the local directory cannot be directly loaded into the Whisper X model. Follow the steps below:

1. Download Tokenizer.json from Hugging-Face directly and move the file to the directory containing the updated model. (e.g. if model is based on Whisper Large V2, get Tokenizer.json [here](#)). Move Tokenizer.json to the folder containing the model that will be converted
2. Run the following command in any terminal:
`ct2-transformers-converter --model {original fine-tuned model path} --output_dir {path to store updated model} --copy_files tokenizer.json processor_config.json --quantization float16` (Read more: [model-conversion](#))
3. Run WhisperX.py on the updated model

The updated model differs as follows:

Original Model	Updated Model
test_whisperx_accept added_tokens.json config.json generation_config.json merges.txt normalizer.json preprocessor_config.json pytorch_model.bin special_tokens_map.json tokenizer_config.json tokenizer.json vocab.json	test_whisperx_accept3 config.json model.bin preprocessor_config.json tokenizer.json vocabulary.json

Uploading Dataset onto Hugging Face for Use

Step 1: Click on Upload Dataset 	Log in into Hugging Face Website [Register a new account if needed] Click on your profile at the top right and you will see this option as shown.
Step 2: Create Github Repository for your dataset 	Self explanatory. You might be asked to link your Github to Hugging Face. Note that you can only view the dataset in the Hugging Face website if the repository is public, unless you pay a fee. You can still use the dataset regardless
Step 3: Add your json/csv file and commit to the repository	Self explanatory as well, just add your data to the Hugging Face hub The data would look something like this if the repository is public:

		Split (2)
		train · 39 rows
		Search this dataset
id	translation	
string · length	dict	
1		
2		
0	{ "yue": "歡迎大家收緊補貼政策", "zh": "歡迎大家收緊《補貼政策》" }	
1	{ "yue": "我想同大家一齊轉嚟下", "zh": "我們同大家一齊轉嚟一下" }	
2	{ "yue": "唔好畀政策局以落成個特區市上上下下", "zh": "不同政策局以至整個特區政府上上下下" }	
3	{ "yue": "過去呢幾年做咩做咁", "zh": "過去幾年的工作表現" }	
4	{ "yue": "睇住期之內係咪真係可以建得咁快咁好所有嘢", "zh": "能否真的趕及住期內完成所有工作" }	
5	{ "yue": "我係黃永我係鍾漢文 Benson 好睇我哋今日電話", "zh": "我係黃永 我係鍾漢文 Benson好, 這次我們" }	
6	{ "yue": "睇好手係睇一份就睇報告睇係唔", "zh": "不只看一份報告, Benson 是" }	
7	{ "yue": "因為過去呢幾日都講好多咁咁咁咁咁", "zh": "因為這幾天已談了很多新措施" }	

Step 4: Use the dataset in code



Yik Hei Kwok
Death052

- Profile
- Account
- Authentication
- Organizations
- Billing
- Access Tokens**
- SSH and GPG Keys
- Webhooks
- Papers

< Create new Access Token

Token type

Fine-grained **Read** Write

This cannot be changed after token creation.

Token name

Token name

This token has read-only access to all your and your orgs comment on discussions.

Create token

Only Applicable if the repository is private

To load a private dataset, use this code in the trainer

```
access_token = "some token"
dataset = load_dataset("Death052/train_test_2", token=access_token)
```

A Read Access token can be generated in
 “Settings→Access Tokens → Create New Token

If the dataset is public, just do this:

```
dataset = load_dataset("Death052/train_test_2")
```

